

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

Giảng viên: Nguyễn Ngọc Giang

Nội dung bài trước

1. Vòng lặp While
2. Vòng lặp For
3. Xử lý chuỗi
4. List (Danh sách)
5. Tuple
6. Range

Nội dung hôm nay

1. Set
2. FrozenSet
3. Dictionary
4. Module và Package
5. Ngoại lệ và xử lý ngoại lệ

Phần 1

Set (tập hợp)

Giới thiệu

- Tập hợp (set) là kiểu dữ liệu đặc sắc trong Python, lấy cảm hứng từ khái niệm tập hợp trong toán học
 - Các đối tượng con đôi một khác nhau: nếu đưa các đối tượng giống nhau vào tập hợp, Python sẽ chỉ giữ lại một
 - Không có tính thứ tự: không thể truy cập đến phần tử con thông qua hệ thống chỉ mục
 - Không phải dữ liệu nào cũng đưa được vào tập hợp: dữ liệu con bắt buộc phải là dạng bất biến (immutable)
 - Thêm: Python sử dụng cấu trúc dữ liệu bảng băm (hashtable) cho tập hợp, đây chính là lý do dữ liệu con phải bất biến để tránh việc dữ liệu bị thay đổi một cách bất lường

Khởi tạo

- Tương tự như danh sách và hàng, khởi tạo tập hợp đơn giản nhất bằng cách liệt kê các phần tử con:
 - Đặt trong cặp ngoặc nhọn {}
 - Ngăn cách bởi phẩy
 - Chú ý: cách này không dùng để khởi tạo tập rỗng (hãy thử xem)

```
>>> basket = {'apple', 'orange', 'apple', 'pear'}  
>>> print(basket)  
{'orange', 'pear', 'apple'}      # xóa phần tử trùng nhau
```

- Tạo tập hợp bằng hàm tạo hoặc các phép toán tập hợp

```
s1 = set([1, 2, 3, 4])      # {1, 2, 3, 4} - copy từ list  
s2 = set((1, 1, 1))        # {1} - copy từ tuple, bỏ lặp  
s3 = s1 - s2                # {2, 3, 4} - hiệu của hai tập  
s4 = set(range(1, 100))    # {1, 2, 3,..., 98, 99}  
s5 = set()                 # {} - tập rỗng
```

Khởi tạo

- Tập hợp cũng có thể khởi bằng bộ suy diễn tập hợp (set comprehension), cú pháp tương tự như danh sách
 - `{ <biểu thức> for <biến> in <tuần tự> }`
 - `{ x for x in 'abracadabra' }`
- Dạng phức tạp hơn: lặp và điều kiện
 - `{ <biểu thức> for <biến> in <tuần tự> if <điều kiện> }`
 - `{ x for x in 'abracadabra' if x not in 'abc' }`
- Dạng phức tạp hơn nữa: lặp và điều kiện rẽ nhánh
 - Trường hợp này phải kết hợp lặp và phép toán if (không dùng rẽ nhánh if được)
 - `{ <A> if <điều kiện> else <B> for <biến> in <tuần tự> }`
 - `{ '?' if x in 'abc' else x for x in 'abracadabra' }`

Khởi tạo

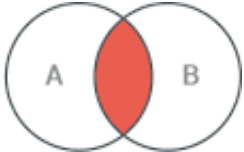
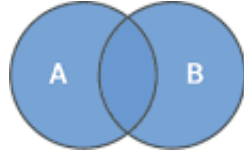
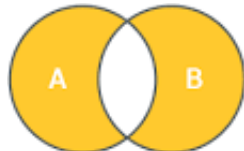
- Bộ suy diễn tập hợp đôi khi khá phức tạp
- Ví dụ: tạo tập hợp chứa mọi hoán vị của chuỗi 'abc'

```
s = {  
    x + y + z  
    for x in 'abc'  
    for y in 'abc'  
    for z in 'abc'  
    if x != y != z != x  
}
```

- Set không thể chứa những đối tượng mutable (có thể bị thay đổi), mặc dù chính set lại có thể thay đổi

```
a = set([1,2], [2,3]) # lỗi  
a = set((1,2), (2,3)) # {(1, 2), (2, 3)}  
a.add("abc")         # {(1, 2), "abc", (2, 3)}
```

Các phép toán trên set

STT	Tên	Kí hiệu	Giải thích	Minh họa
1	Phép giao	&	Lấy phần chung của hai tập	
2	Phép hợp	 	Lấy phần gộp của hai tập	
3	Phép hiệu	-	Lấy phần riêng của một tập	
4	Phép loại	^	Lấy phần khác (loại bỏ phần chung)	
5	Kiểm tra tồn tại	in	Trả về True nếu phần tử nằm trong tập hợp	
6	Kiểm tra không thuộc	not in	Trả về True nếu phần tử không thuộc tập hợp	

Các phép toán trên set

STT	Tên	Kí hiệu	Giải thích
7	So sánh “bằng”	==	Trả về True nếu hai tập giống nhau
8	So sánh “khác”	!=	Trả về True nếu có ít nhất một phần tử thuộc tập này mà không thuộc tập kia
9	So sánh “lớn hơn”	>	Trả về True nếu mọi phần tử của tập thứ hai đều có trong tập thứ nhất và có ít nhất một phần tử thuộc tập thứ nhất không xuất hiện trong tập thứ hai
10	So sánh “lớn hơn hoặc bằng”	>=	Trả về True nếu tập thứ nhất bằng hoặc lớn hơn tập thứ hai
11	So sánh “nhỏ hơn”	<	Trả về True nếu mọi phần tử thuộc tập thứ nhất đều có trong tập thứ hai và có ít nhất một phần tử thuộc tập thứ hai không xuất hiện trong tập thứ nhất
12	So sánh “nhỏ hơn hoặc bằng”	<=	Trả về True nếu tập thứ nhất bằng hoặc nhỏ hơn tập thứ hai

Các phép toán trên set

```
a = set('abracadabra')      # {'d', 'r', 'c', 'b', 'a'}  
b = set('alacazam')        # {'z', 'c', 'm', 'l', 'a'}
```

Phép Hiệu: thuộc a nhưng không thuộc b

```
print(a - b)                # {'r', 'd', 'b'}
```

Phép Hợp: thuộc a hoặc b

```
# {'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
```

```
print(a | b)
```

Phép Giao: thuộc cả a và b

```
print(a & b)                # {'a', 'c'}
```

Phép Xor: thuộc hoặc a, hoặc b nhưng không phải cả 2

```
# {'r', 'd', 'b', 'm', 'z', 'l'}
```

```
print(a ^ b)
```

Duyệt các phần tử của tập hợp

- Cách đơn giản nhất là duyệt bằng vòng for

```
a = { 10, 2, 3, 1, 3, 2, 10 }
```

```
for e in a:
```

```
    print(e)
```

- Cần thận: thứ tự in kết quả của vòng for không nhất thiết phải theo thứ tự các phần tử đưa vào tập hợp
- Trường hợp cần duyệt theo chỉ mục, sử dụng kết hợp với hàm enumerate:

```
a = { 10, 2, 3, 1, 3, 2, 10 }
```

```
for index, value in enumerate(a):
```

```
    print(index, value)
```


Các phương thức của set

- Một số phương thức thường hay sử dụng
 - `add(e)`: thêm e vào tập hợp
 - `clear()`: xóa mọi phần tử trong tập hợp
 - `copy()`: tạo một bản sao của tập hợp
 - `difference(x)`: tương đương với phép trừ đi x
 - `difference_update(x)`: loại bỏ những phần tử trong x khỏi tập (cập nhật vào tập hiện tại)
 - `discard(e)`: bỏ e khỏi tập
 - `intersection(x)`: tương đương với phép giao với x
 - `intersection_update(x)`: tương đương với phép giao với x (cập nhật vào tập hiện tại)

Các phương thức của set

- Một số phương thức thường hay sử dụng
 - `isdisjoint(x)`: trả về True nếu tập không có phần chung nào với x
 - `issubset(x)`: trả về True nếu tập là con của x, tương đương với phép so sánh $\leq x$
 - `issuperset(x)`: trả về True nếu x là tập con của tập, tương đương với phép so sánh $\geq x$
 - `pop()`: lấy một phần tử ra khỏi tập (không biết trước)
 - `remove(e)`: bỏ e khỏi tập, báo lỗi nếu không tìm thấy e
 - `symmetric_difference(x)`: tương đương với phép $\wedge x$
 - `symmetric_difference_update(x)`: tương đương với phép $\wedge x$ (cập nhật vào tập hiện tại)
 - `union(x)`: tương đương với phép hợp với x
 - `update(x)`: đưa các phần tử của tập x vào tập hiện tại

Phần 8

Frozenset (tập hợp tĩnh)

Frozenset (tập tĩnh)

- Frozenset giống set, nhưng không thể bị thay đổi

```
b = frozenset(((1,2), (2,3))) # {(1, 2), (2, 3)}  
b.add("abc")                 # lỗi
```

- Sự khác biệt giữa tập tĩnh và tập hợp:
 - Tập tĩnh là kiểu bất biến, tập hợp là kiểu khả biến (dù các thành phần của nó phải là bất biến)
 - Tập tĩnh thường nhanh và ít tốn bộ nhớ hơn tập hợp
 - Tập tĩnh không sử dụng được các phương thức thay đổi nội dung như `add()` hoặc `remove()`
 - Tập tĩnh vẫn hỗ trợ các phép toán của kiểu tập hợp (giao, hợp, hiệu,...)
 - Tập tĩnh vẫn hỗ trợ các phép so sánh với tập hợp
 - Tập tĩnh có thể dùng làm khóa cho dữ liệu từ điển (học sau)

Tổng kết lại

- Python có kiểu dữ liệu tập hợp (**set**) lấy cảm hứng từ tập hợp trong toán học. Set có hai đặc điểm chính:
 - Các dữ liệu con bên trong nó đôi một khác nhau
 - Chỉ chứa các dữ liệu loại bất biến (immutable)
- Tập hợp không có tính thứ tự, vì vậy không có phép toán chỉ mục và cắt lát, tuy vậy vẫn có thể duyệt các phần tử con trong tập hợp bằng **for**
- Python cung cấp nhiều phương thức, phép so sánh và phép toán hữu ích cho tập hợp
- Tập tĩnh (frozenset) là tập hợp bất biến, không thể thay đổi sau khi khởi tạo xong

Bài tập

1. Tạo một tập hợp gồm các phần tử từ 0 đến 99, in chúng ra màn hình
2. Tạo một tập hợp gồm các số nguyên lẻ trong khoảng từ 1 đến 199, in chúng ra màn hình
3. Tạo một tập hợp gồm các số nhập vào từ bàn phím (nhập trên 1 dòng, cách nhau bởi ký tự trống), tìm và in ra số phần tử của tập, giá trị lớn nhất và nhỏ nhất trong tập
4. Nhập vào từ bàn phím họ và tên đầy đủ của các sinh viên trong lớp, mỗi người trên một dòng. Việc nhập sẽ kết thúc khi người dùng gõ vào dòng trống. Sau đó, hãy in ra các họ và các tên của sinh viên trong lớp

Bài tập

5. Nhập số nguyên N , tạo một tập hợp các số nguyên dương d là ước số của N
6. Nhập 2 số nguyên a và b , hãy tạo ra một tập hợp các số d là ước số chung của cả a và b
7. Nhập một dãy số nguyên từ bàn phím, các số được viết liên tiếp, ngăn cách nhau bởi dấu chấm phẩy (;), hãy đếm xem dãy nhập vào có bao nhiêu số khác nhau
8. Vé Vietlott Mega là bộ 6 số chỉ từ 01 đến 45. Người chơi sẽ thắng nếu chọn đúng ít nhất 5 trong 6 số, thứ tự không quan trọng. Hãy viết chương trình nhập vào N bộ 6 số của N người, sau đó nhập tiếp 6 số của giải đặc biệt và in ra các bộ số của người chơi thắng cuộc.

Bài tập

9. Một công ty có 3 phòng chức năng có dùng thể dùng chung nhân viên là phòng nhân sự, phòng hành chính và phòng truyền thông. Các nhân viên có mã nhân viên là các số nguyên dương. Hãy thực hiện các việc:
- Nhập danh sách mã nhân viên của cả 3 phòng, danh sách được viết liên tục trên một dòng, ngăn cách bởi dấu phẩy
 - Ba phòng ban này sử dụng bao nhiêu nhân viên?
 - In ra danh sách các mã nhân viên thuộc cả 3 phòng
 - In ra danh sách các mã nhân viên chỉ thuộc 1 phòng
 - Tìm cặp phòng dùng chung nhiều nhân viên nhất, nếu có nhiều cặp phòng như vậy thì in ra tất cả các cặp
 - Với từng phòng, in ra mã nhân viên đầu tiên của phòng (có mã nhỏ nhất)



Phần 2

Dictionary (từ điển)

Dictionary (từ điển)

- Từ điển trong cuộc sống: các mục từ tra cứu ngữ nghĩa
 - Các mục từ được sắp thứ tự
 - Các mục từ thường khác nhau, một số từ điển cho phép các mục từ lớn chia thành nhiều mục từ con
- Dictionary trong Python lấy cảm hứng từ từ điển trong cuộc sống:
 - Một mục là một cặp (pair) khóa (**key**) và giá trị (**value**)
 - Tương đương với khái niệm mục từ và ngữ nghĩa trong từ điển thông thường
 - Các khóa (key) không được trùng nhau, như vậy có thể xem từ điển như một loại set
 - Các khóa không sắp thứ tự như từ điển thông thường
 - Chỉ dữ liệu bất biến (immutable) mới được dùng làm khóa

Dictionary (từ điển)

- Một cặp trong từ điển viết ở dạng `<key> : <value>`
- Từ điển có thể khai báo trực tiếp dùng cú pháp của `set`

```
d1 = { }                # từ điển rỗng
d2 = { 1: 'one', 2: 'two' } # từ điển các cặp số-chuỗi
d3 = { 'one': 1, 'two': 2 } # từ điển các cặp chuỗi-số
d4 = { 'tên': 'nam', 'sdt': 0 } # từ điển hỗn hợp
```

- Như vậy Python coi từ điển là dạng mở rộng của tập hợp
- Trường hợp lấy dữ liệu từ nguồn khác, cách thích hợp nhất là sử dụng hàm khởi tạo `dict()`

```
d5 = dict(d4)           # lấy dữ liệu từ d4
print(d5)                # {'tên': 'nam', 'sdt': 0}
d6 = dict()              # tạo từ điển rỗng
print(d6)                # {}
```

Dictionary (từ điển)

- Python cũng cho phép tạo từ điển bằng bộ suy diễn từ điển, với cú pháp tương tự như bộ suy diễn danh sách
- Ví dụ: tạo từ điển gồm các bộ khóa là số tự nhiên nhỏ hơn N và giá trị tương ứng là lập phương của nó

```
n = int(input('N = '))  
d = { i: i * i * i for i in range(n) }  
print(d)
```

- Ví dụ: tạo từ điển có khóa các ký tự xuất hiện trong từ S và giá trị là số lần xuất hiện của ký tự đó trong S

```
S = input('N = ')  
d = { w : S.count(w) for w in S }  
print(d)
```

Dictionary (từ điển)

- Chú ý: chỉ những loại dữ liệu immutable (không thể thay đổi) mới có thể dùng làm key của từ điển

```
dic = { (1,2,3): "abc", 3.1415: "abc" }
```

```
dic = { [1,2,3]: "abc" }           # lỗi
```

- Một số phép toán / phương thức thường dùng
 - `len(d)`: trả về độ dài của từ điển (số cặp key-value)
 - `del d[k]`: xóa key k (và value tương ứng)
 - `k in d`: trả về True nếu có key k trong từ điển
 - `k not in d`: trả về True nếu không có key k trong từ điển
 - `pop(k)`: trả về value tương ứng với k và xóa cặp này đi
 - `popitem()`: trả về (và xóa) một cặp (key, value) tùy ý

Dictionary (từ điển)

- Một số phép toán / phương thức thường dùng
 - `get(k)`: lấy về value tương ứng với key k
 - Khác phép `[]` ở chỗ `get` trả về `None` nếu `k` không phải là key
 - `update(w)`: ghép các nội dung từ từ điển `w` vào từ điển hiện tại (nếu key trùng thì lấy value từ `w`)
 - `items()`: trả về list các cặp (key, value)
 - `keys()`: trả về các key của từ điển
 - `values()`: trả về các value của từ điển
 - `pop(k)`: trả về value tương ứng với `k` và xóa cặp này đi
 - `popitem()`: trả về (và xóa) một cặp (key, value) tùy ý

Dictionary (từ điển)

- Dùng zip để ghép 2 list thành từ điển

```
>>> l1 = ["a", "b", "c"]
```

```
>>> l2 = [1, 2, 3]
```

```
>>> c = zip(l1, l2)
```

```
>>> for i in c:
```

```
...     print(i)
```

```
...
```

```
('a', 1)
```

```
('b', 2)
```

```
('c', 3)
```


Truy xuất dữ liệu theo khóa

- Từ điển cho phép lấy giá trị tương ứng với khóa k bằng phương thức `get(k)`, nhưng có thể dùng chỉ mục thuận tiện hơn nhiều

```
d = { 1: 'one', 2: 'two', 3: 'three' }
print('d.get(3) = ', d.get(3))          # d.get(3) = three
print('d[3] = ', d[3])                  # d[3] = three
print('d.get(9) = ', d.get(9))          # d.get(9) = None
print('d[9] = ', d[9])                  # KeyError
d[3] = 'ba'                             # cập nhật cặp 3: 'three' thành 3: 'ba'
d[4] = 'bốn'                             # thêm mới cặp 4: 'bốn'
print(d)                                # {1: 'one', 2: 'two', 3: 'ba', 4: 'bốn'}
```

- Chú ý: hai cách này không tương đương. Khi dữ liệu không có trong từ điển, `get(k)` trả về `None` còn chỉ mục phát sinh ngoại lệ `KeyError`

Xóa dữ liệu trong từ điển

- Python có nhiều cách để xóa dữ liệu trong từ điển hoặc lấy dữ liệu ra khỏi từ điển
- Các phương thức pop, popitem, clear được thiết kế cho những tình huống lập trình khác nhau

```
d = { 1: 'one', 3: 'three', 2: 'two', 0: 'zero' }  
del d[1]           # xóa bỏ mục 1: 'one'  
print(d)           # {3: 'three', 2: 'two', 0: 'zero'}  
print(d.pop(2))    # 'two'  
print(d.popitem()) # (0, 'zero')  
d.clear()          # xóa rỗng từ điển  
print(d)           # {}
```

Kiểm tra dữ liệu trong từ điển

- Phép kiểm tra dữ liệu trong từ điển chỉ làm việc với khóa

```
dic = { 1: 'one', 3: 'three', 2: 'two', 0: 'zero' }
```

```
# kiểm tra có 2 trong từ điển không? True
```

```
print(2 in dic)
```

```
# kiểm tra không có 'one' trong từ điển phải không? True
```

```
print('one' not in dic)
```

```
# kiểm tra cặp (3, 'three') có trong từ điển không? False
```

```
print((3, 'three') in dic)
```

Duyệt dữ liệu trong từ điển

- Python cung cấp nhiều phương thức lấy về các nhóm dữ liệu trong từ điển, ta có thể dễ dàng duyệt chúng với vòng lặp **for**

```
d = { 1: 'one', 3: 'three', 2: 'two', 0: 'zero' }  
# duyệt từ điển theo khóa: 1, 3, 2, 0  
for i in d: print(i)  
# duyệt từ điển theo khóa: 1, 3, 2, 0  
for i in d.keys(): print(i)  
# duyệt theo cặp khóa-giá trị: (1, 'one'), (3, 'three'),..  
for i in d.items(): print(i)  
# duyệt theo giá trị: 'one', 'three', 'two', 'zero'  
for i in d.values(): print(i)
```

Bài tập

Bạn được cung cấp dữ liệu về quyền truy cập của người dùng trong một hệ thống phần mềm. Mỗi người dùng có thể truy cập vào một số module nhất định. Yêu cầu:

1. Liệt kê tất cả các module có trong hệ thống và danh sách người có thể truy cập module đó.
2. Tìm người dùng có quyền truy cập vào tất cả các module.
3. Tìm các cặp người dùng có quyền truy cập giống hệt nhau.
4. Đếm số lượng nhóm quyền truy cập duy nhất.

```
users_access = {  
    "Alice": {"dashboard", "settings", "reports"},  
    "Bob": {"dashboard", "settings", "reports"},  
    "Carol": {"dashboard", "analytics"},  
    "David": {"dashboard", "settings", "reports", "analytics"},  
    "Eve": {"dashboard", "reports"},  
}
```



Phần 3

Module và Package

Module (khởi)

- Trong python, file mã nguồn được xem là một module
 - Có phần mở rộng .py
 - Mọi hàm, biến, kiểu trong file là các thành phần của module
- Sử dụng module:
 - Có thể sử dụng các thành phần trong các module khác bằng cách import (nhập/nạp) module đó, đây là phương pháp cơ bản để tái sử dụng lại mã nguồn
 - Cú pháp: `import <tên-module>`
 - Có thể import cùng lúc nhiều module cách nhau bởi dấu phẩy
 - Nếu muốn sử dụng các hàm, biến trong module thì cần viết tường minh tên module đó
 - Có thể import riêng một hoặc nhiều hàm từ một module, cú pháp: `from <tên-module> import fuc1, fuc2,... fucN`

Package (gói)

- Package = Thư mục các module (lưu trữ vật lý trên ổ đĩa)

```
import numpy
A = array([1, 2, 3])           # lỗi
A = numpy.array([1, 2, 3])    # ok
import numpy as np
B = np.array([1, 2, 3])       # ok
from numpy import array
C = array([1, 2, 3])          # ok
```

- Module và Package giúp quản lý tốt hơn mã nguồn
- Nhóm các hàm, biến, lớp xử lý cùng một chủ đề, giúp phân cấp và sử dụng dễ dàng hơn
- Giải quyết tranh chấp định danh của thư viện khác nhau
- Python có rất nhiều các package hỗ trợ mọi nhu cầu xử lý

Module math

- Một module rất thông dụng của python: `import math`
- Math có nhiều hằng số định nghĩa sẵn:
 - `pi`: 3.141592...
 - `e`: 2.718281...
 - `tau`: 6.283185... ($2 * \pi$)
 - `inf`: dương vô cùng (âm vô cùng là `-math.inf`)
 - `nan`: not a number (tương đương với `float('nan')`)
- Math chứa nhiều hàm toán học:
 - `ceil(x)`: trả về số nguyên nhỏ nhất nhưng không nhỏ hơn x
 - `copysign(x, y)`: copy dấu của y gán sang x
 - Ví dụ: `copysign(1.0, -0.0)` trả về -1
 - `fabs(x)`: trả về trị tuyệt đối của x

Module math

- Math chứa nhiều hàm toán học (tiếp...):
 - `factorial(x)`: trả về $x!$
 - `floor(x)`: trả về số nguyên lớn nhất nhưng không vượt quá x
 - `gcd(a, b)`: trả về ước số chung lớn nhất của a và b
 - `isinf(x)`: trả về True nếu x là dương/âm vô cùng
 - `isnan(x)`: trả về True nếu x là NaN (not a number)
 - `trunc(x)`: trả về phần nguyên của x
 - `exp(x)`: trả về e^x
 - `log(x[, y])`: trả về $\log_y x$, mặc định $y = e$
 - `log10(x)`: trả về $\log_{10} x$
 - `pow(x, y)`: trả về x^y
 - `sqrt(x)`: trả về $\sqrt{x}$

Module math

- Math cung cấp một số hàm lượng giác:
 - `degrees(x)`: chuyển x từ radians sang độ
 - `radians(x)`: chuyển x từ độ sang radians
 - `acos(x)`: trả về arc cos x (độ đo radians)
 - `asin(x)`: trả về arc sin x (độ đo radians)
 - `atan(x)`: trả về arc tang x (độ đo radians)
 - `cos(x)`: trả về cos x (độ đo radians)
 - `sin(x)`: trả về sin x (độ đo radians)
 - `tan(x)`: trả về tang x (độ đo radians)

Tổng kết

- Kiểu dữ liệu từ điển là kiểu dữ liệu lấy cảm hứng từ từ điển trong cuộc sống
 - Từ điển là một tập các mục, một mục là một cặp key và value
 - Các key phải đôi một khác nhau, phải là kiểu dữ liệu bất biến
 - Có thể tra cứu value thông qua phép toán chỉ mục với key
 - Dữ liệu trong từ điển không có tính thứ tự, nhưng có thể duyệt bằng vòng lặp
- Module và Package là cơ chế Python sử dụng để kiểm soát mã nguồn hiệu quả hơn
 - Một file mã nguồn = một module, có thể tái sử dụng bởi import
 - Một thư mục chứa mã nguồn = một package
- Module math chứa khá nhiều các hàm toán học cơ bản

Bài tập

1. Cho D là từ điển định nghĩa cách đọc các chữ số ở tiếng Anh, hãy in ra các value của D theo thứ tự tăng dần.
2. Nhập một từ điển D, hãy in ra các value khác nhau trong từ điển.
3. Nhập một từ điển D có các value là các số nguyên, hãy in ra màn hình 3 giá trị value lớn nhất.
4. Nhập một string S, hãy tạo từ điển D trong đó key là các chữ xuất hiện trong S còn value tương ứng là số lần xuất hiện các chữ đó trong S

Ví dụ: S = “dai hoc thuy loi”

$$D = \{ 'd':1, 'a':1, 'i':2, ' ':3, 'h':2, \\ 'o':2, 'c':1, 't':1, 'u':1, 'y':1, 'l':1 \}$$

Bài tập

5. Nhập từ điển `prices` lưu trữ giá của các loại trái cây và từ điển `stock` lưu trữ số lượng tồn của từng loại. Sau đó hãy in ra thứ tự các loại trái cây còn trong cửa hàng giảm dần theo tổng giá trị của từng loại.

Ví dụ:

Dữ liệu nhập vào cho ra từ điển như sau:

- `prices = { "banana": 4, "apple": 2, "orange": 1.5, "pear": 3 }`
- `stock = { "banana": 6, "orange": 32, "pear": 15 }`

Kết quả in ra thứ tự:

orange	48
pear	45
banana	24
apple	0

Bài tập

6. Tạo ra một từ điển lưu lượng mưa trung bình trong các tháng từ năm 2000 đến 2019. Quy cách như sau:
- Từ điển có 12 mục, khóa của mỗi mục là một tháng
 - Giá trị ứng với khóa là danh sách 20 số đại diện cho 20 năm
 - Lượng mưa là số thực ngẫu nhiên từ 100 đến 4000
7. Nhập 2 từ điển A có N cặp (key, value) và B có M cặp (key, value). Từ A và B hãy tạo từ điển C theo quy tắc sau:
- Một mục trong C thì key phải xuất hiện trong A hoặc B
 - Nếu key chỉ xuất hiện trong A hoặc trong B thì value là value tương ứng trong A (hoặc B)
 - Nếu key xuất hiện trong cả A và B thì value là max của value tương ứng trong A và B



Phần 4

Ngoại lệ và xử lý ngoại lệ?

Nội dung

1. Ngoại lệ là gì?
2. Xử lý ngoại lệ
3. Một số loại ngoại lệ thường gặp
4. Tự sinh ngoại lệ
5. Bài tập

1. Ngoại lệ là gì?

- Việc một chương trình máy tính hoạt động không hoàn hảo là không thể tránh khỏi
- Thường thì giới lập trình chia lỗi thành 3 nhóm
 1. **Lỗi khi viết chương trình**: hệ quả là chương trình không chạy được khi gặp dòng lệnh sai, nếu là thông dịch (hoặc không dịch được, nếu là biên dịch)
 2. **Lỗi khi chương trình chạy**: hệ quả là phải thực hiện lại
 - Chẳng hạn như nhập liệu không đúng, thì phải nhập lại
 3. **Ngoại lệ**: vẫn là lỗi, xảy ra khi có một bất thường và khiến một chức năng không thể thực hiện được
 - Chẳng hạn như đang ghi dữ liệu ra một file, nhưng file đó lại bị một tiến trình khác xóa mất
- Ngoại lệ = lỗi? Đúng, nhưng không hẳn

1. Ngoại lệ là gì?

- Ranh giới giữa ngoại lệ và lỗi khá mong manh, thậm chí khó phân biệt trong nhiều tình huống
- Vấn đề: Cách chia lỗi thành 3 nhóm có khuynh hướng cho rằng môi trường thực thi của chương trình là thân thiện và hoàn hảo
- Python quan điểm nên chia lỗi thành 2 loại
 - **Syntax error**: viết sai cú pháp, khiến chương trình thông dịch không dịch được, trong trường hợp này lập trình viên phải viết lại mã, không có phương án nào khác
 - **Exception**: xảy ra bất thường không như thiết kế
 - Như vậy xử lý exception sẽ khiến chương trình ổn định và hoạt động tốt trong mọi tình huống
 - Trường hợp này lập trình viên phải có phương án khắc phục

1. Ngoại lệ là gì?

- Ví dụ về syntax error:

```
>>> while True print('Hello world')
```

```
File "<stdin>", line 1
```

```
    while True print('Hello world')
```

```
                ^
```

```
SyntaxError: invalid syntax
```

- Ví dụ về exception:

```
>>> 10 * (1/0)
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ZeroDivisionError: division by zero
```

- Có vẻ như syntax error cũng chỉ là một exception!!!

2. Xử lý ngoại lệ

```
while True:
```

Vòng lặp nhập X
cho đến khi người dùng
nhập vào đúng giá trị số

```
try:
```

```
x = int(input("Nhập số X: "))  
break
```

Khởi nhập X
(có thể nhập lỗi)

```
except ValueError:
```

```
print("Lỗi, hãy nhập lại.")
```

Xử lý khi lỗi xảy ra

```
print("X =", x)
```

Cú pháp try-except-else-finally

- Cú pháp: try:
 except:
 else:
 finally:
- Công việc của từng khối:
 - Khối “try”: đoạn mã có khả năng gây lỗi, khi lỗi xảy ra, khối này sẽ bị dừng ở dòng gây lỗi
 - Khối “except”: đoạn mã xử lý lỗi, chỉ thực hiện nếu có lỗi xảy ra
 - Khối “else”: có thể xuất hiện ngay sau khối except cuối cùng, đoạn mã sẽ được thực hiện nếu không có except nào được thực hiện (đoạn try không có lỗi)
 - Khối “finally”: còn được gọi là khối clean-up, luôn được thực hiện dù có xảy ra lỗi hay không

Cú pháp try-except-finally

■ Chú ý:

- Khối **try** chỉ có 1 khối duy nhất, phải viết đầu tiên
- Khối **finally** có thể có hay không, nếu có thì khối này phải viết cuối cùng
- Khối **except** có thể không viết, có một khối, hoặc nhiều khối except khác nhau (để xử lý nhiều tình huống lỗi khác nhau)
- Một khối **except** có thể xử lý một loại lỗi, nhiều loại lỗi hoặc tất cả các loại lỗi
- Nếu không xử lý triệt để lỗi có thể “ném” trả lại lỗi này bằng lệnh “**raise**”
- Có thể phát sinh một ngoại lệ bằng lệnh “**raise** <lỗi>”
- Khi lỗi xảy ra, một biến chứa lỗi được phát sinh và “ném” xuống phần các khối **except**, khối **except** đầu tiên “bắt” được ngoại lệ sẽ xử lý nó

Quy tắc “bắt” ngoại lệ: lọt sàng xuống nia

```
except (NameError, TypeError):      # xử lý 2 loại lỗi
    print("Name or Type error")

except IOError as e:                # lấy biến lỗi, đặt tên là e
    print(e)
    raise                            # “ném” trả lại lỗi này

except ValueError:                  # xử lý lỗi Value
    print("Value error")

except:                              # xử lý mọi lỗi khác
    print("An error occurred")
    raise NameError("Ko bit")        # tạo ra một lỗi “Ko bit”

else:                                # thực hiện nếu không có lỗi
    print("OK")
```

Nhiệm vụ của khối finally: xử lý dự phòng

- Việc chấm dứt đoạn mã một cách bất ngờ (trong khối **try** hoặc **except**) đem lại một số vấn đề
 - Chẳng hạn như trong khối **try** ta tạo một tập tin tạm thời để chứa dữ liệu trung gian, dùng xong sẽ xóa tập tin đó đi
 - Khi đoạn mã chấm dứt đột ngột vì ngoại lệ: lệnh xóa tập tin tạm không thực hiện
 - Hậu quả: sau một thời gian sử dụng chương trình, máy tính của người dùng có xuất hiện rất nhiều tập tin “rác”
 - Thực tế vấn đề này đã xảy ra với phần mềm Microsoft Word
- Khối **finally**: còn được gọi là khối “dọn dẹp” (clean-up)
 - Python đảm bảo rằng đoạn mã trong khối luôn được thực hiện dù có ngoại lệ xảy ra hay không
 - Trong ví dụ trên: ta sẽ đặt phần mã xóa tập tin tạm vào **finally**

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
SystemExit	Xảy ra khi chương trình gọi hàm sys.exit()
KeyboardInterrupt	Xảy ra khi người dùng nhấn phím ngắt để cố gắng kết thúc chương trình (thường là Ctrl-C hoặc Delete)
GeneratorExit	Xảy ra khi chương trình đóng một bộ sinh (generator) bằng hàm close() Chú ý: đây không thực sự là một lỗi
Exception	Lớp cha của mọi lớp lỗi, ngoại trừ KeyboardInterrupt , SystemExit và GeneratorExit
StopIteration	Xảy ra khi cố gắng đọc đối tượng tiếp theo từ iterator, nhưng khi đó iterator đã ở cuối của kiểu tuần tự nên không có đối tượng cần đọc
StopAsyncIteration	Xảy ra khi cố gắng đọc đối tượng tiếp theo từ một iterator bất đồng bộ, nhưng khi đó iterator chưa đồng bộ được đối tượng tiếp theo

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
ArithmeticError	Lớp cha của mọi lỗi xử lý tính toán
FloatingPointError	Lớp con của ArithmeticError (<i>hiện đã bỏ không dùng, chỉ giữ lại để tương thích với các mã cũ</i>)
OverflowError	Lớp con của ArithmeticError , xảy ra khi kết quả của phép toán số học quá lớn hoặc quá nhỏ đến mức không thể biểu diễn
ZeroDivisionError	Lớp con của ArithmeticError , xảy ra khi thực hiện phép chia hoặc phép đồng dư với số 0
AssertionError	Xảy ra khi câu lệnh kiểm tra điều kiện assert thất bại
AttributeError	Xảy ra khi không thể gán thuộc tính hoặc tham chiếu
BufferError	Xảy ra khi không thể tạo vùng đệm hoặc bị tràn vùng đệm

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
EOFError	Xảy ra khi cố gắng đọc thêm dữ liệu mặc dù đã đến cuối tập tin (không thể đọc thêm được gì nữa)
ImportError	Xảy ra khi chương trình khai báo sử dụng thành phần trong thư viện (một hàm, một biến hoặc một lớp) nhưng không tìm thấy thành phần này
ModuleNotFoundError	Kiểu con của ImportError , xảy ra khi chương trình khai báo sử dụng một thư viện nhưng Python không thể nạp được thư viện đó trên máy hiện tại (thường do không tìm thấy mã máy của thư viện đó)
LookupError	Lớp cha của các lỗi xảy ra khi tìm kiếm dữ liệu
IndexError	Kiểu con của LookupError , xảy ra khi giá trị chỉ mục (index) nằm ngoài phạm vi của một biến kiểu tuần tự (string, list, tuple,...)
KeyError	Kiểu con của LookupError , xảy ra khi không tìm thấy khóa trong từ điển

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
MemoryError	Xảy ra khi hết bộ nhớ nên chương trình không thể tiếp tục thực thi hoặc bộ nhớ bị phân mảnh đến mức không thể tạo được các biến cần thiết để tiếp tục thực thi chương trình
NameError	Xảy ra khi cố gắng truy cập một biến không tồn tại
UnboundLocalError	Kiểu con của NameError , xảy ra khi tham chiếu đến một biến cục bộ nhưng biến cục bộ đó không tồn tại
ReferenceError	Xảy ra khi tham chiếu bị lỗi, chẳng hạn như truy cập một biến hoặc một thuộc tính đã bị bộ dọn rác (garbage collector) xóa đi
RuntimeError	Xảy ra một lỗi thực thi chung chung, không rơi vào nhóm các lỗi đã biết
NotImplementedError	Kiểu con của RuntimeError , xảy ra khi chương trình cố gắng thực thi một phương thức hoặc một hàm, nhưng vì lý do nào đó phần thân hàm (phương thức) chưa được viết

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
OSError	Xảy ra khi thực hiện một hàm liên quan đến hệ thống (chẳng hạn như đọc ghi đĩa) nhưng gặp lỗi. <i>Để giữ tương thích với các phiên bản trước, lỗi này còn có các tên khác như IOError, EnvironmentError và WindowsError</i> Đây là một lớp lớn cung cấp nhiều thông tin lỗi khi làm việc với hệ điều hành, các lớp con của lớp này gồm: - BlockingIOError - ChildProcessError - ConnectionError ○ BrokenPipeError ○ ConnectionAbortedError ○ ConnectionRefusedError ○ ConnectionResetError - FileExistsError - FileNotFoundError - InterruptedError - IsADirectoryError - NotADirectoryError - PermissionError - ProcessLookupError - TimeoutError

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
RecursionError	Kiểu con của RuntimeError , xảy ra khi gọi đệ quy quá nhiều lớp (độ sâu quá lớn)
SyntaxError	Xảy ra khi cố gắng chạy một lệnh viết sai cú pháp
IndentationError	Kiểu con của SyntaxError , xảy ra khi cố gắng chạy một lệnh viết thụt lề không chính xác
TabError	Kiểu con của IndentationError , xảy ra khi thụt lề sử dụng các dấu tab và dấu space không nhất quán
SystemError	Xảy ra khi trình thông dịch gặp các lỗi nội bộ
TypeError	Xảy ra khi chương trình cố gắng chuyển một đối tượng sang một kiểu khác nhưng không phù hợp (chẳng hạn cố gắng đổi một chuỗi tên riêng sang dạng số nguyên)

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
ValueError	Xảy ra khi hàm hoặc phương thức hoặc phép toán nhận được một đối số có kiểu đúng nhưng giá trị không phù hợp
UnicodeError	Kiểu con của ValueError , xảy ra khi có lỗi liên quan đến quá trình xử lý dữ liệu unicode
UnicodeEncodeError	Kiểu con của UnicodeError , xảy ra khi lỗi liên quan đến quá trình mã hóa dữ liệu
UnicodeDecodeError	Kiểu con của UnicodeError , xảy ra khi lỗi liên quan đến quá trình giải mã dữ liệu
UnicodeTranslateError	Kiểu con của UnicodeError , xảy ra khi lỗi liên quan đến quá trình chuyển đổi dữ liệu (kể cả chuyển đổi code page)

3. Một số loại ngoại lệ thường

Ngoại lệ	Lý do gây ra
Warning	Lớp cha của các loại cảnh báo, những lỗi nhẹ hoặc có tiềm năng xảy ra trong tương lai. Các lớp con của lớp này gồm: - UserWarning - DeprecationWarning - PendingDeprecationWarning - SyntaxWarning - RuntimeWarning - FutureWarning - ImportWarning - UnicodeWarning - BytesWarning - ResourceWarning

4. Tự sinh ngoại lệ với từ khóa raise

- Python cung cấp từ khóa raise sử dụng khi cần phát sinh một ngoại lệ
 - Nếu chỉ viết “raise”: cách viết này chỉ đúng trong khối **except**, khi ta không xử lý được ngoại lệ hiện tại và “ném trả” ngoại lệ về cho chương trình cha
 - Nếu viết “raise <biến>”: phát sinh một ngoại lệ và <biến> sẽ chứa các thông tin báo lỗi về ngoại lệ xảy ra
 - Trong trường hợp này, <biến> nên có kiểu **Exception** hoặc kế thừa từ **Exception**, kiểu càng cụ thể thì càng cung cấp nhiều thông tin cho quá trình sửa lỗi
 - Lập trình viên có thể tạo một kiểu ngoại lệ mới, kĩ thuật này sử dụng hướng đối tượng là chủ đề nằm ngoài bài giảng này nên sẽ không được đề cập tới

Ví dụ về phát sinh ngoại lệ

```
try:
    a = int(input("Nhập một số nguyên dương nhỏ hơn 100: "))
    # sinh lỗi khi số quá bé
    if a <= 0:
        raise ValueError("Bạn đã nhập một số quá nhỏ")
    # sinh lỗi khi số quá lớn
    if a >= 100:
        raise ValueError("Bạn cần nhập số nhỏ hơn 100")
# bắt lỗi:
# - nhập không phải số nguyên
# - nhập số quá lớn
# - nhập số quá bé
except ValueError as ex:
    print(ex)
```

Bài tập

1. Viết chương trình nhập 2 số nguyên a và b , sau đó tính và in ra giá trị phân số a/b . Chú ý xử lý ngoại lệ trong các tình huống dưới đây:
 - Người dùng nhập a hoặc b không phải số nguyên
 - Người dùng nhập $b = 0$
2. Viết chương trình yêu cầu người dùng nhập a , b và c là độ dài 3 cạnh của một tam giác. Xử lý ngoại lệ trong các tình huống sau:
 - Người dùng nhập a , b hoặc c không phải là kiểu số
 - Người dùng nhập giá trị 0 hoặc số âm cho a , b hoặc c
 - Người dùng nhập a , b , c dương nhưng không thể là cạnh của một tam giác

Bài tập

3. Viết chương trình cho phép người dùng nhập liên tiếp các số nguyên dương, yêu cầu:

- Việc nhập liệu sẽ kết thúc bình thường (không exception) khi người dùng nhập số 0
- Phát sinh exception và thông báo “Lỗi số âm!!!” khi người dùng nhập một số nguyên âm bất kỳ
- Phát sinh exception và thông báo “Lỗi nhập số” khi người dùng nhập vào chuỗi không phải số nguyên
- Phát sinh exception và thông báo “Lỗi nhập lặp” khi người dùng nhập 4 số nguyên dương liên tiếp giống nhau
- Phát sinh exception và thông báo “Lỗi nhập chẵn” khi người dùng nhập 5 số chẵn liên tiếp
- Sau khi phát sinh exception, xử lý và thông báo, chương trình cần tiếp tục hoạt động như bình thường

